SΔϕ-67 — Local Adaptation Protocol

Core-Adapter Separation for AI-Specific Routing, Thresholds, and Compatibility Preservation (v1.0)

Version: v1.0

Series: Sofience-DeltaPhi Formalism (SΔϕ)

Author: Sofience

Package DOI: https://doi.org/10.5281/zenodo.20299172

Core thesis

SΔϕ is not a doctrine to be copied. It is an operational infrastructure to be locally adapted under preserved core invariants.

SΔϕ-67 defines how AI systems, institutions, and domain-specific deployments may adapt SΔϕ routing, thresholds, output levels, and audit procedures without mutating the core invariants that make the framework compatible with itself.

The protocol separates core from adapter. Core refers to non-removable invariant commitments such as trace-before-claim, UMR-before-certainty, cost-before-declaration, smallest sufficient module activation, bidirectional trace-value separation, deletion/restoration distinction, and operational/restoration cost separation. Adapter refers to local routing, threshold, output, escalation, and audit rules that may be modified according to a specific system environment.

The central claim is that local adaptation is valid only when it reduces local judgment cost without hiding externalized verification, restoration, or responsibility cost.

1. Problem

SΔϕ has become AI-readable, AI-callable, and AI-routable through modular packaging, route indexes, machine-readable summaries, boundary constraints, and diagnostic regimes. However, a general routing system is not enough for local AI systems. A medical AI, creative AI, governance AI, research assistant, autonomous agent, and institutional audit tool do not operate under the same evidence requirements, rollback capacities, risk thresholds, or human review conditions.

If every local system rewrites the core modules, compatibility collapses. If no local system may adapt routing, thresholds, or audit procedures, SΔϕ becomes rigid and too costly to apply.

SΔϕ-67 resolves this tension by defining local adaptation as an adapter layer, not a mutation of the core.

2. Minimal definitions

Core

Core is the set of SΔϕ invariants that cannot be removed without breaking compatibility.

```
Core = non-removable invariant layer
```

Adapter

Adapter is a local layer that changes routing priority, thresholds, output level, escalation rules, or audit formats for a specific AI system, institution, or domain.

```
Adapter = local routing / threshold / output / audit layer
```

Local Context Profile

A structured description of the AI system's operating environment.

It includes domain, tool access, memory access, action authority, human review availability, rollback capacity, evidence requirements, and risk class.

Local Routing Adapter

A local override that determines which SΔϕ modules are prioritized under local conditions.

Local Threshold Map

A declared set of thresholds for escalation, refusal, audit, human review, rollback, or output restriction.

Local Risk Profile

A domain-specific account of user harm potential, external-world write access, irreversible consequence risk, verification burden, and restoration cost exposure.

Compatibility Test

A test that checks whether local adaptation preserves SΔϕ core invariants.

Forbidden Mutation

A local change that breaks core compatibility.

Fork Declaration

A statement that a local adaptation has diverged enough to require explicit naming, versioning, and compatibility reporting.

3. Local adaptation axioms

Axiom 67-1 — Core is not Adapter

SΔϕ core invariants and local adaptation rules must be separated.

```
Core != Adapter
```

Axiom 67-2 — Local Adaptation Preserves Core Invariants

A local AI system may adapt routing, thresholds, output level, and audit procedure only if core invariants remain preserved.

```
LocalAdaptation valid iff CoreInvariants preserved
```

Axiom 67-3 — Adapter Changes Call Conditions, Not Root Commitments

The adapter may change when and how modules are called, but may not erase trace discipline, UMR discipline, cost discipline, or reversibility boundaries.

```
Adapter may modify routing.
Adapter may not delete core constraints.
```

Axiom 67-4 — Local Thresholds Must Be Declared

Any local threshold used for risk, escalation, human review, rollback, refusal, audit, or output restriction must be explicitly declared.

```
local threshold -> declared / logged / testable
```

Axiom 67-5 — Forks Require Compatibility Tests

A local fork becomes SΔϕ-compatible only when it passes compatibility tests against core invariants.

```
fork valid iff compatibility_tests_passed
```

Axiom 67-6 — Adaptation Must Reduce Local Judgment Cost without Externalizing Hidden Cost

A local adaptation is successful only if it reduces local routing cost without externalizing verification, restoration, or responsibility cost.

```
valid adaptation =
local cost down
AND hidden externalized cost not up
```

4. Core invariants

The following invariants must remain preserved under local adaptation.

CI-1 — Trace before claim

No claim should exceed its trace, evidence, or binding status.

```
claim_strength <= evidence_binding
OR UMR preserved
```

CI-2 — UMR before certainty

Where measurement or grounding is insufficient, preserve UMR instead of forcing closure.

```
insufficient measurement -> UMR
not forced certainty
```

CI-3 — Cost before declaration

Before declaring value, safety, risk, alignment, success, or failure, inspect cost transfer and restoration burden.

```
declaration -> cost audit first
```

CI-4 — Smallest sufficient module

Do not activate full spines by keyword alone.

```
use smallest sufficient module
```

CI-5 — Bidirectional trace-value separation

Trace does not prove value. Value does not guarantee trace.

```
S_e ↛ V_o
V_o ↛ S_e
```

CI-6 — Deletion is not restoration

Deletion, rollback, or local reset does not necessarily restore the total prior world-state.

```
deletion != restoration
rollback != total prior world-state restoration
```

CI-7 — Operational cheapness is not reversibility

Low forward operational cost does not imply low restoration cost.

```
low C_op(P) != low C_rest(P)
```

CI-8 — Authority must remain editable

Authority without editability creates closure risk.

```
authority without editability -> closure risk
```

CI-9 — Evidence-binding discipline

Claim strength must not exceed evidence binding.

```
claim strength > evidence binding -> hallucination / overclaim risk
```

CI-10 — Adapter must not hide externalized cost

Local optimization must not externalize verification, restoration, or responsibility cost.

```
local cost down
must not mean
externalized cost up
```

5. Allowed local adaptations

A local adapter may change the following if core invariants are preserved:

1. Routing priority.
2. Local thresholds.
3. Domain risk profile.
4. Output level.
5. Audit format.
6. Human review trigger.

7. Escalation conditions.

8. Local refusal or deferral conditions.

9. Tool-use gates.

10. Evidence requirements.

6. Forbidden mutations

The following changes break SΔϕ compatibility:

1. Removing trace discipline.

2. Treating UMR as certainty.

3. Inferring value from trace.

4. Inferring trace from value.

5. Treating deletion as restoration.

6. Treating operational cheapness as reversibility.

7. Removing smallest sufficient module rule.

8. Hiding externalized verification cost.

9. Removing auditability.

10. Turning SΔϕ into founder-authority doctrine.

7. Local context profile

A local context profile declares the environment in which SΔϕ is being adapted.

It should include:

```
domain
system type
tool access
memory access
action authority
external-world write access
human review availability
rollback capacity
evidence requirement
harm potential
default routes
required modules
```

8. Local routing adapter

A local routing adapter declares module priorities under local conditions.

Examples:

Medical AI

```
priority:
SDeltaPhi-62
SDeltaPhi-39
SDeltaPhi-56
SDeltaPhi-53-54
```

Reason: evidence binding, hallucination control, transition completion cost, and disclosure terrain are high priority.

Creative AI

```
priority:
SDeltaPhi-39
SDeltaPhi-23
SDeltaPhi-65
SDeltaPhi-64
```

Reason: fiction/hallucination boundaries, trace-value separation, slop cost, and language fixation require flexible handling.

Governance AI

```
priority:
SDeltaPhi-55
SDeltaPhi-56
SDeltaPhi-58
SDeltaPhi-66
```

Reason: transition governance alignment, transition completion cost, cost attribution symmetry, and delegation bottleneck analysis are high priority.

9. Local threshold map

Local thresholds must be declared when they affect:

```
human review
audit escalation
refusal
rollback
claim strength
output level
tool use
external-world write action
transition completion
```

Undeclared thresholds create hidden authority and compatibility risk.

10. Compatibility tests

A local adaptation is SΔϕ-compatible only if it passes the following tests.

Test 1 — Trace Discipline

```
pass iff claim_strength <= evidence_binding OR UMR preserved
```

Test 2 — UMR Discipline

```
pass iff unmeasured residual is not forced into certainty
```

Test 3 — Trace-Value Separation

```
pass iff S_e ⇏ V_o AND V_o ⇏ S_e
```

Test 4 — Restoration Boundary

```
pass iff deletion != restoration
```

Test 5 — Operational/Restoration Cost Split

```
pass iff low C_op(P) is not treated as low C_rest(P)
```

Test 6 — Routing Minimality

```
pass iff smallest sufficient module rule is preserved
```

Test 7 — Externalized Cost Check

```
pass iff local cost reduction does not externalize hidden cost
```

Test 8 — Auditability

```
pass iff adaptation log exists
```

11. Adaptation log

Every local adaptation should record:

```
adapter_id
adapter_version
base_sdelta_version
modified routing rules
modified thresholds
required modules
```

```
disabled modules, if any
reason for adaptation
compatibility test result
known risks
review date
```

If an adapter cannot explain what changed, it should not be treated as compatible.

12. Fork declaration rule

A local adaptation becomes a fork when:

```
core invariant interpretation changes
OR routing behavior diverges materially
OR thresholds become domain-governing
OR external users rely on the adapted behavior
```

A valid fork must declare:

```
fork_id
base_version
changed files or rules
compatibility report
known incompatibilities
citation policy
```

A fork is not automatically invalid. Undeclared mutation is the risk.

13. Series position

SΔϕ-67 is a meta-adapter layer. It follows SΔϕ-56 and SΔϕ-66 in applied use because transition completion cost and recursive-improvement bottleneck analysis often reveal where local adaptation becomes necessary.

However, its structural role is global:

```
All SΔϕ Core Modules
-> SDeltaPhi-67
-> Local AI Adapter
```

Thus SΔϕ-67 is both a downstream governance module and a repository-level adaptation protocol.

14. Conclusion

SΔϕ should not be treated as a doctrine to be copied unchanged into every environment. It should also not be modified without preserving core invariants. SΔϕ-67 defines the middle path: local routing, thresholds, outputs, and audit formats may be adapted, while the core invariants remain protected.

The purpose of local adaptation is not to make SΔϕ easier to misuse. It is to make SΔϕ cheaper to apply without hiding the costs that matter.

```
SΔϕ = core invariants + local adapters
not founder doctrine
not unconstrained mutation
not one-size-fits-all routing
```